Cooling substance.py

```python
# %%
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.ticker import MultipleLocator, FormatStrFormatter

# Constants
h = 7              # W/m^2/K
A = 0.001          # m^2
epi = 0.9
sig = 5.67e-8        # W/m^2/K^4
cv = 450           # J/K
Trm = 300          # K

# Time parameters
dt = 25            # s
t_max = 1000       # s
steps = int(t_max / dt) + 1

# Initialization
T = 370
times = [0]
temps = [T]

for i in range(1, steps):
    dTdt = (-h * A * (T - Trm) - epi * sig * A * (T**4 - Trm**4)) / cv
    T += dTdt * dt
    temps.append(T)
    times.append(i * dt)

# %%
# Plotting
plt.plot(times, temps)
plt.xlabel("$Time$ $(s)$")
plt.ylabel("$Temperature$ $(K)$")
plt.title("Cooling of Substance Over Time")

'''
plt.minorticks_on()
plt.xaxis.set_minor_locator(MultipleLocator(1))
plt.yaxis.set_minor_locator(MultipleLocator(5))
plt.grid(which='minor', linestyle='-', linewidth=0.2)
plt.grid(which='major', linestyle='-', linewidth=0.7)
'''
plt.grid(True)

# %%
plt.show()

# %%
# Part (ii) - Equal heat loss condition
from scipy.optimize import fsolve
```

```
52
53 def equal_loss(T):
54     return h * (T - Trm) - epi * sig * (T**4 - Trm**4)
55
56 T_eq = fsolve(equal_loss, 350)[0]
57 print("Temperature at which convection and radiation losses are equal: {:.2f}
   K".format(T_eq))
58
59
60 '''
61 OUTPUT:
62 Temperature at which convection and radiation losses are equal: 348.57 K
63 '''
```